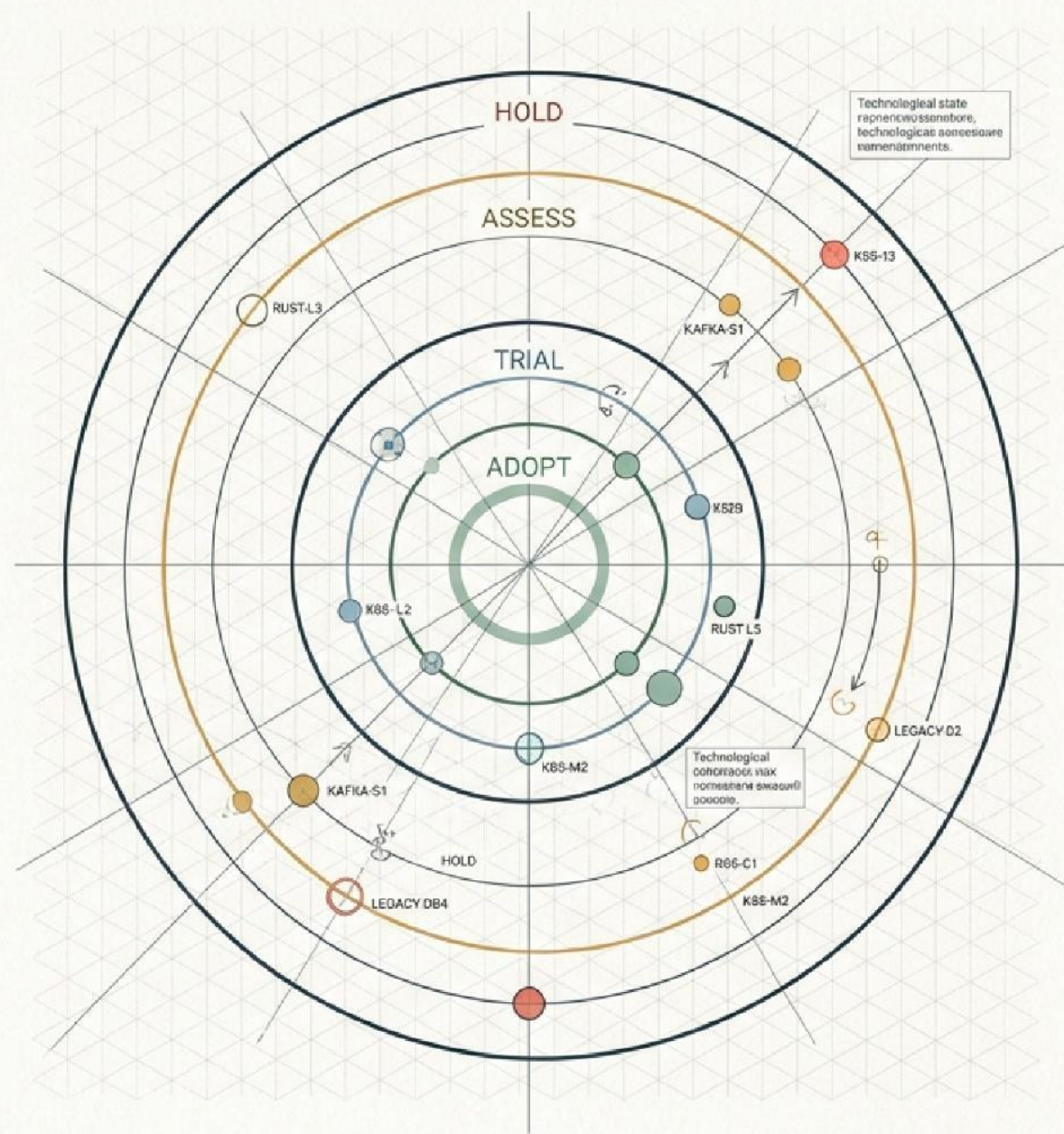


Технологический радар и Эволюция архитектуры

Стратегическое руководство по управлению инновациями, контролю техдолга и долгосрочному планированию систем.





Проверить, идет ли запись

Меня хорошо видно
& слышно?





Иван Четвериков

AI Architect

- Более 5 лет опыта в разработке и архитектуре
- Специалист в разработке и проектировании AI-приложений в Raft

 @istrebitel_1

 istrebitel.3.12@gmail.com

Правила вебинара



Активно
участвуем



Off-topic обсуждаем
в учебной группе



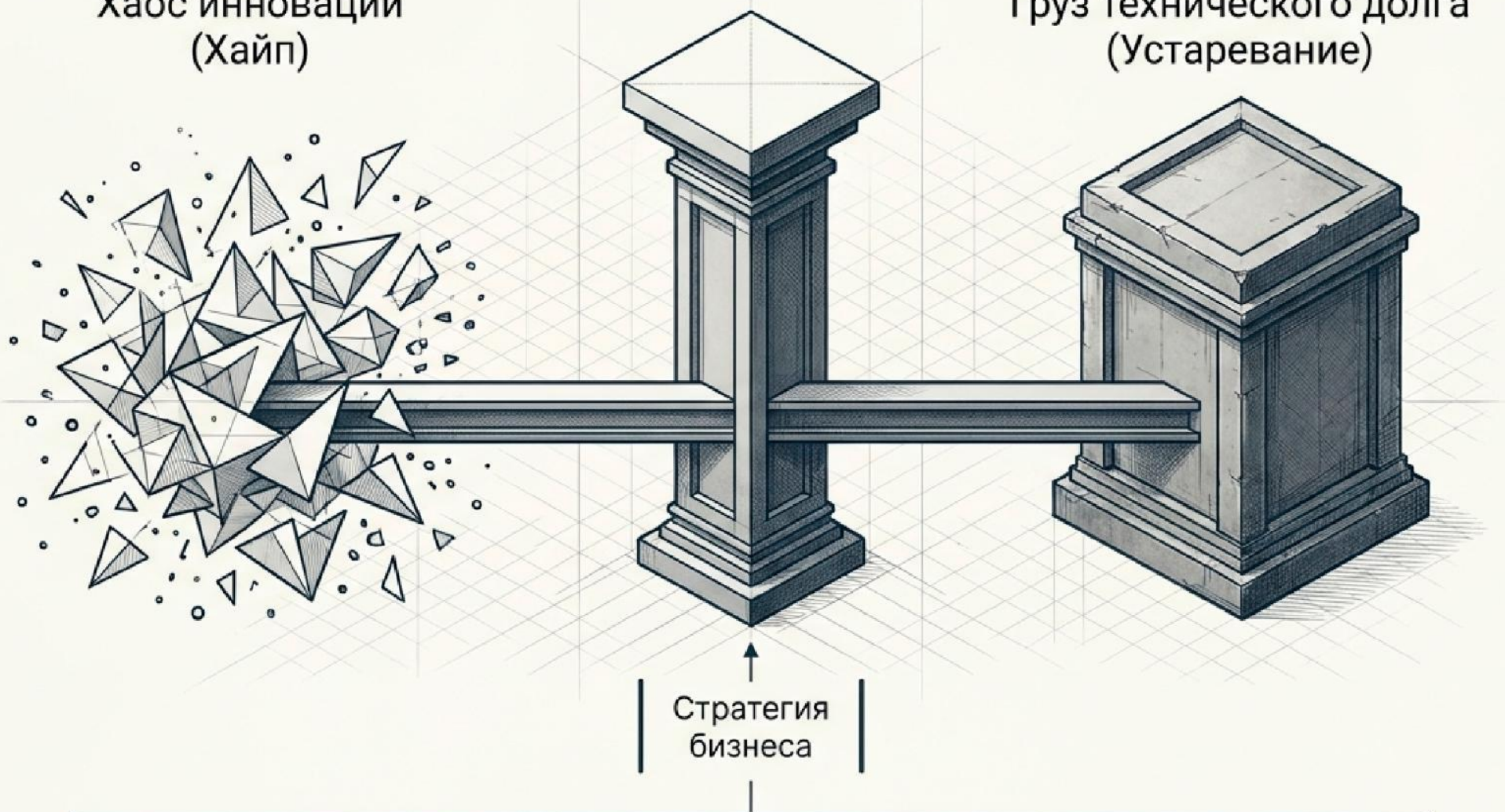
Задаем вопрос
в чат или голосом



Вопросы вижу в чате,
могу ответить не сразу

Хаос инноваций
(Хайп)

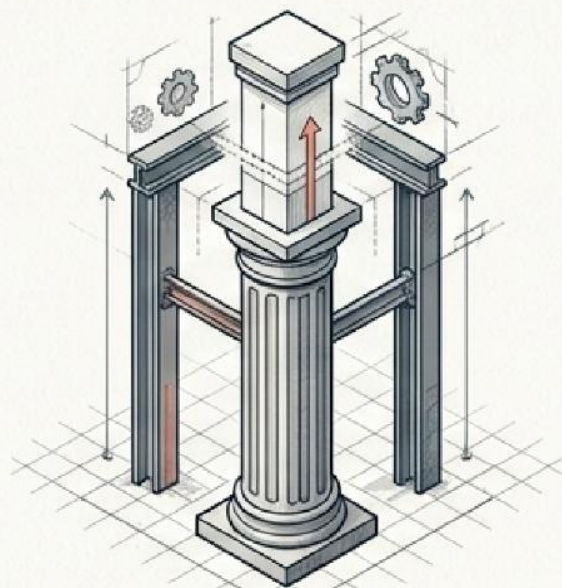
Груз технического долга
(Устаревание)



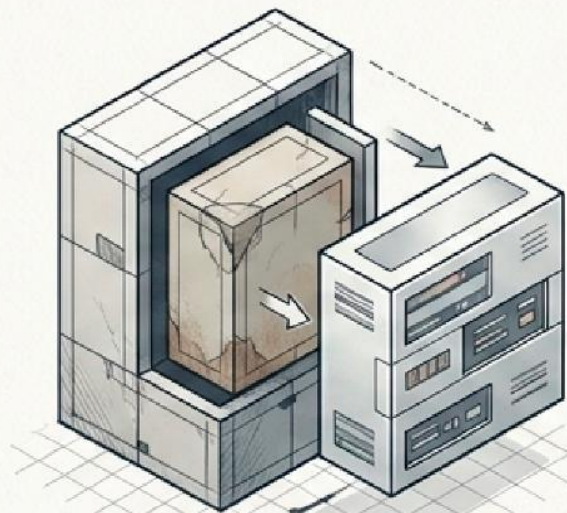
Архитектор находится между необходимостью внедрять новое для роста бизнеса и необходимостью поддерживать старое для его стабильности.

Принципы эволюционной архитектуры

Эволюционная архитектура поддерживает непрерывные изменения как фундаментальный принцип проектирования, а не как реакцию на кризис.



1. Долгосрочное планирование.
Изменение системы без её
полного разрушения.



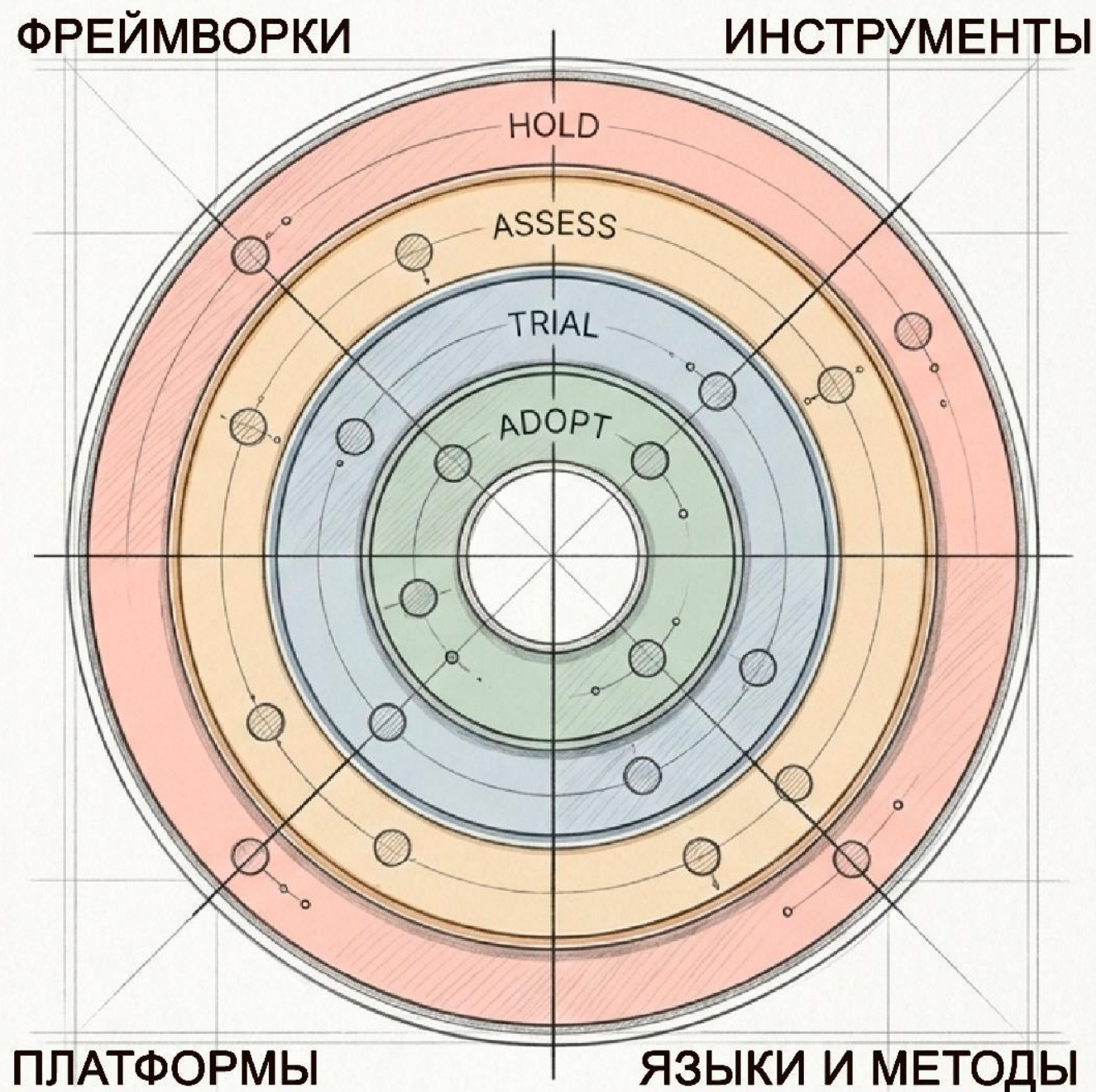
2. Предотвращение устаревания.
Стратегия постепенной
модернизации компонентов.



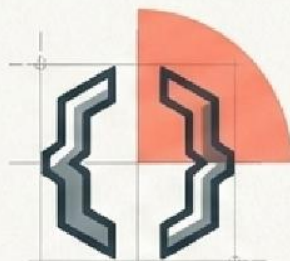
3. Связь с бизнесом. Архитектура
развивается вместе с продуктом
(Архитектура = Продукт).

Инструмент стратега: Технологический радар

Это визуальный фреймворк, который помогает систематизировать хаос новых библиотек и инструментов, превращая интуитивные догадки в измеримую стратегию развития.



Анатомия Радара: Квадранты (Где мы ищем?)



Фреймворки

Примеры: React, Spring



Инструменты

Примеры: Docker, GitHub Actions



Платформы

Примеры: AWS, Kubernetes



Языки и методы

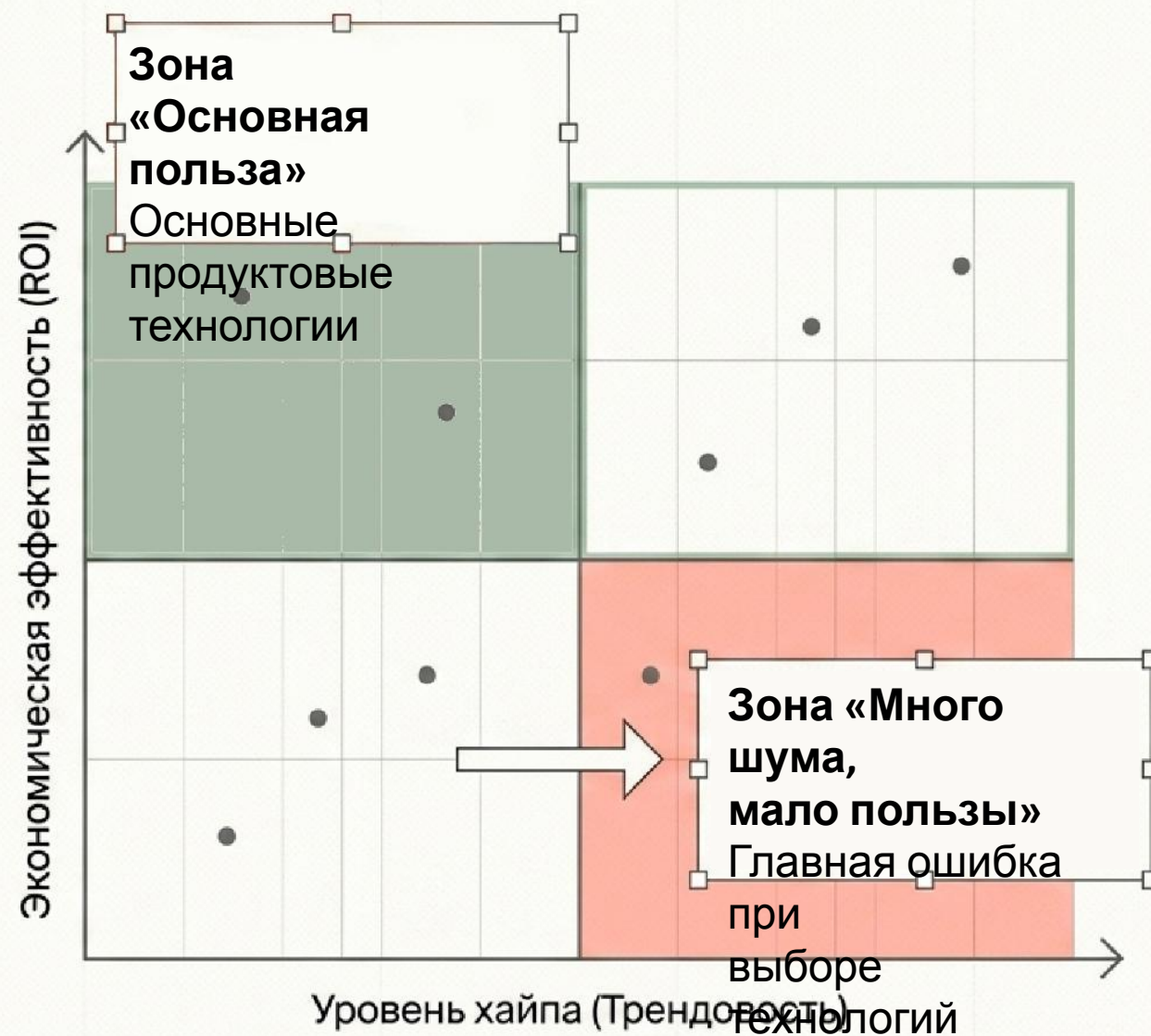
Примеры: Go, Rust, Micro-frontends

Матрица зрелости: Кольца радара

Кольцо	Уровень инвестиций	Риск / Характеристика
 Adopt (Внедрение)	Инвестируем уверенно	Низкий риск. Высокая ценность. Стандарт по умолчанию.
 Trial (Проба)	Тестируем на проекте	Средний риск. Потенциал понятен, нужен реальный опыт.
 Assess (Оценка)	Читаем доклады, PoC	Неизвестный риск. Изучаем концепт, влияние пока неясно.
 Hold (Удержание)	Замораживаем инвестиции	Накопление техдолга. Выводим из эксплуатации, ищем замену.

Фильтр решений: Экономика против Хайпа

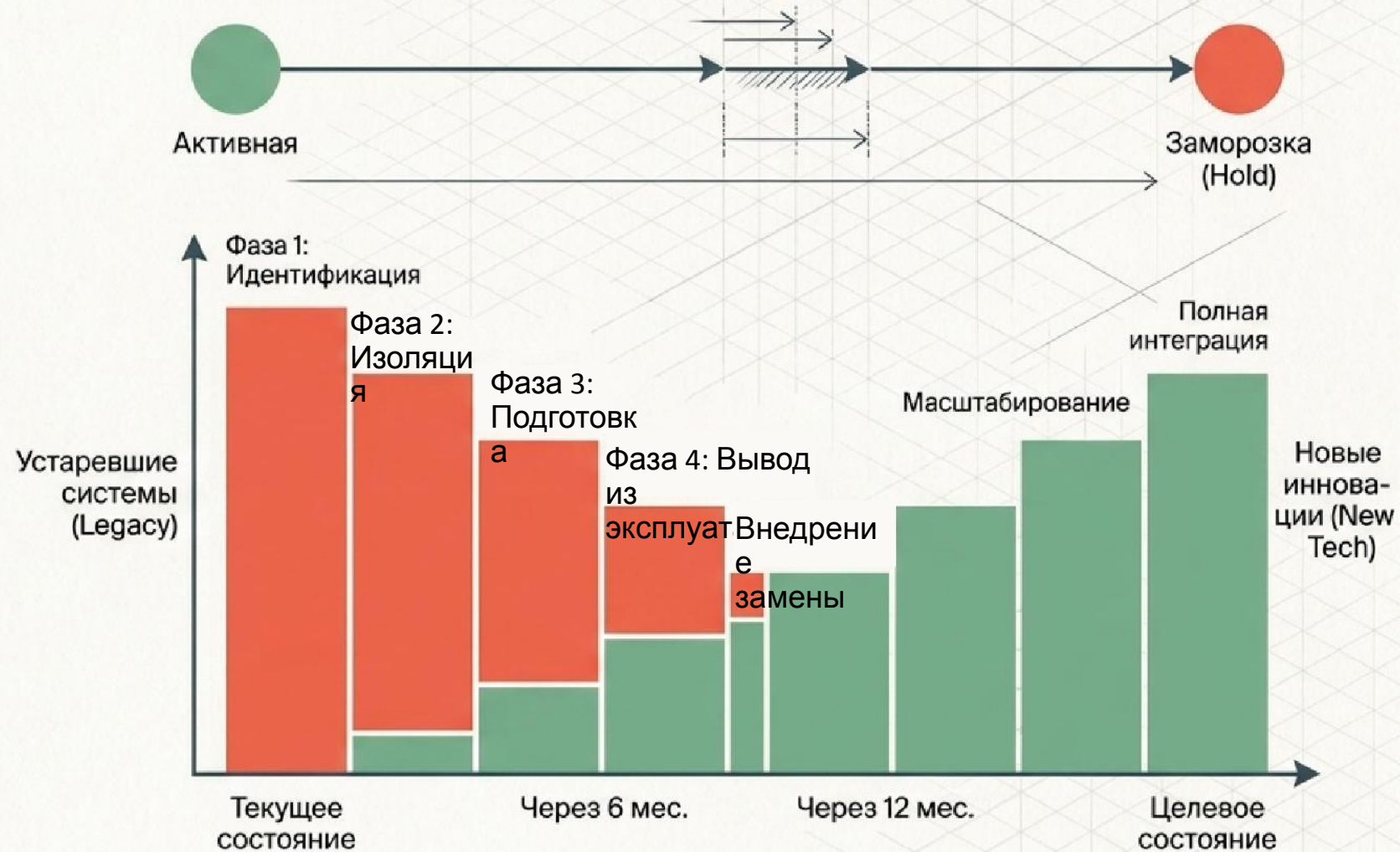
Любая новая технология должна оцениваться по её способности снижать издержки, ускорять Time-to-Market или открывать новые рынки для бизнеса.



Управление техническим долгом (Зона Hold)

Радар работает не только на добавление, но и на удаление. Сектор Hold — это ваш бэклог по выплате технического долга.

Заморозка инвестиций в устаревшие технологии высвобождает критические ресурсы для внедрения инноваций и предотвращает устаревание всей системы.

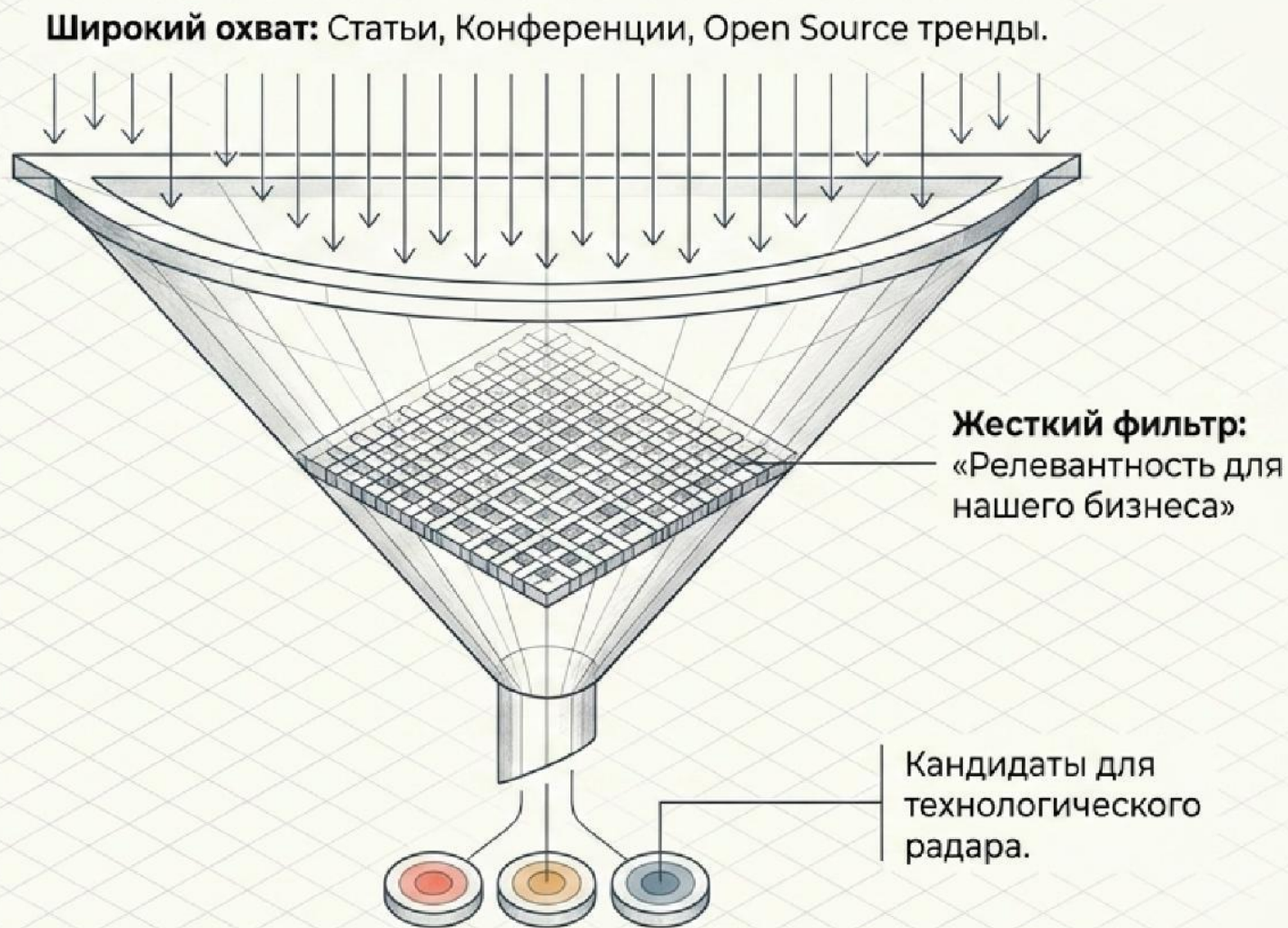


Протокол внедрения инноваций

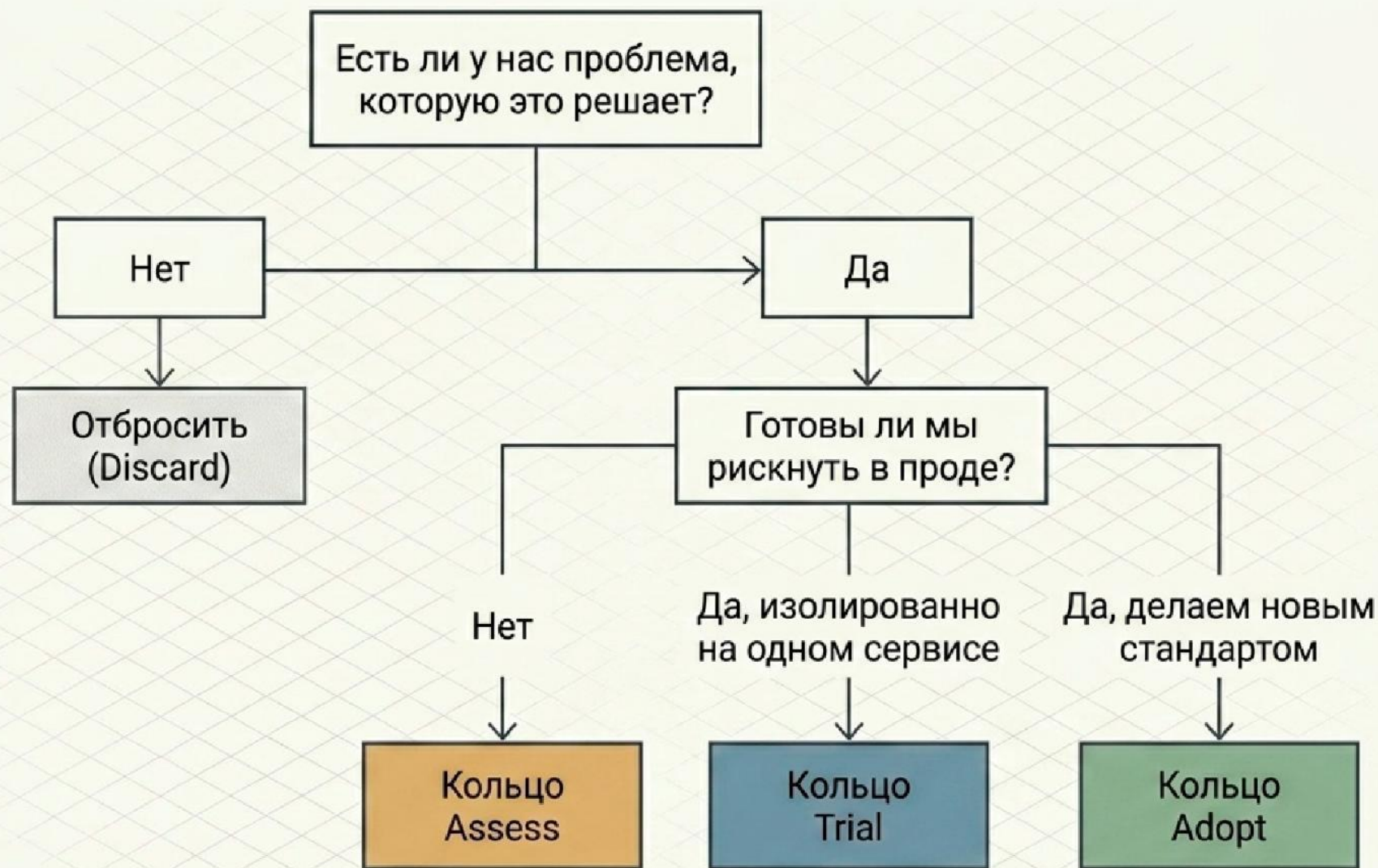
Создание радара — это командный процесс.
Алгоритм его проведения состоит из 3 этапов:



Этап 1. Сбор данных и скан рынка

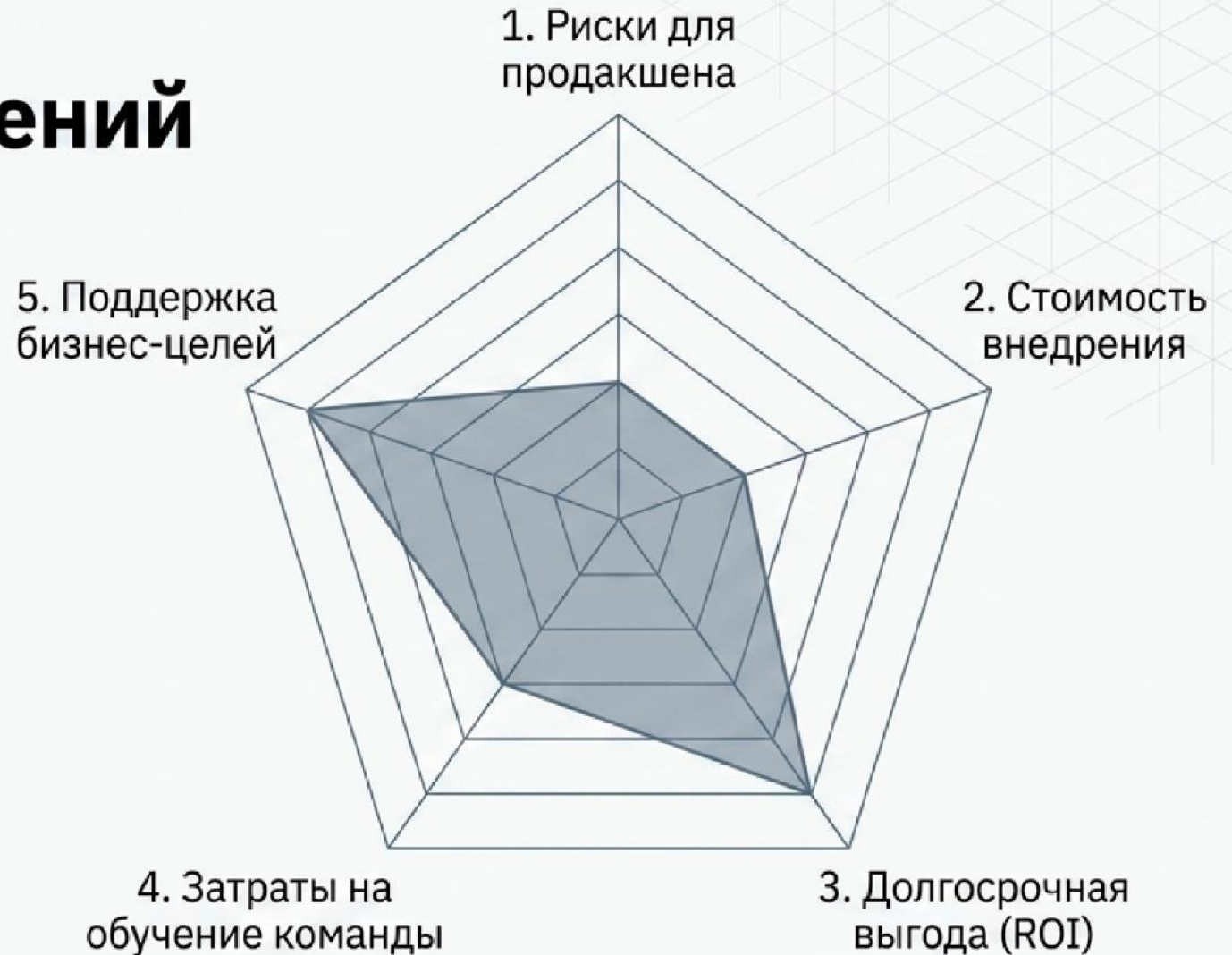


Этап 2. Позиционирование (Искусство дебатов)



Этап 3. Защита архитектурных решений

Любое перемещение в зону Adopt требует защиты решения. Архитектор выступает как лидер, аргументирует не «красотой решения», а снижением бизнес-рисков, окупаемостью или стоимостью поддержки.



Синтез: Цикл архитектурной эволюции



Стратегическое резюме



Эволюционируй системно.

Планируйте изменения как норму, а не как реакцию на кризис устаревания.



Фильтруй экономикой.

Используйте радар, чтобы отсекай технологический хайп, не приносящий бизнес-ценности (ROI).



Управляй техдолгом.

Сектор Hold – ваш главный инструмент для планового вывода устаревших систем из эксплуатации.



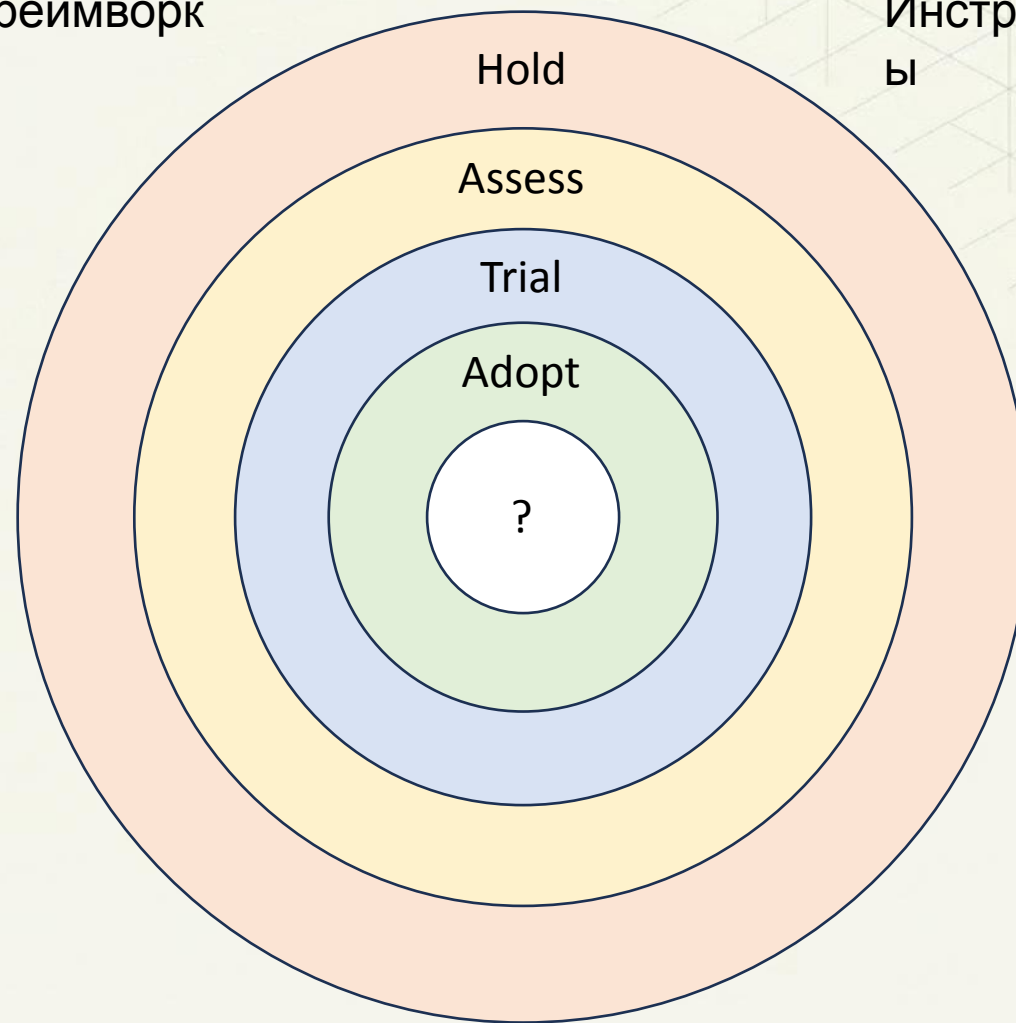
Ваша очередь: Запуск процесса

Технологический радар вашего текущего проекта.

- Какие 3 новые технологии вы добавите в зону Assess?
- Какую устаревшую технологию вы обязаны отправить в зону Hold уже завтра?

Фреймворк
и

Инструмент
ы



Платформ
ы

Языки и
методы



Legacy project - Текущий стек проекта (AS-IS)

Проект по промышленной автоматизации. Стек Python, Java, Docker, React. Модели описаны и обучены на python, обернуты в API на FastAPI и собраны в Docker контейнеры. Java управляет бекендом и объединяет разрозненные контейнеры в общую логику. Фронт написан на React.

Проект представляет собой гетерогенную систему, где разные части отвечают за разные уровни автоматизации:

- **ML-слой:** Python (FastAPI), упакованный в Docker-контейнеры. Используется для инференса обученных моделей.
- **Бэкенд-оркестратор:** Java (управляет логикой взаимодействия контейнеров и бизнес-процессами).
- **Фронтенд:** React (интерфейсы операторов и аналитические панели).
- **Инфраструктура:** Docker (вероятно, запуск через Docker Compose или на отдельных VM).

Нам необходимо распределить технологии по квадрантам (Языки, Инструменты, Платформы, Методы) и кольцам зрелости (Hold, Assess, Trial, Adopt), чтобы управлять техдолгом и внедрять

Применение технологического радар для трансформации

Фреймворки

Hold

Assess

Trial

Adopt

FastAPI - Остается стандартом для ML-сервисов

Инструменты

Hold

Assess

Микросервисная архитектура на Go. Оценка перехода от монолитной логики на Java к легковесным сервисам на Go, которые лучше утилизируют ресурсы

Trial

GitOps (например, ArgoCD). Метод доставки изменений в Kubernetes для автоматизации деплоя в промышленном контуре.

Adopt

Платформы

Hold

Standalone Docker / Docker Compose. Управление разрозненными контейнерами вручную признается устаревшим методом, ведущим к накоплению техдолга.

Assess

Trial

Kubernetes (K8s). Мы переходим к внедрению K8s на одном из участков производства как пилотного сервиса с последующим переходом на стадию Adopt.

Adopt

Языки и методы

Hold

Java. В данном кейсе Java перемещается в зону Hold для написания новой логики оркестрации. Это означает, что мы поддерживаем старый код, но запрещаем развитие системы на этом языке

Assess

Go (Golang). Мы начинаем изучать Go как замену Java для высоконагруженных микросервисов. На этом этапе проводятся PoC и оценка того, как Go справится с параллельной обработкой данных

Trial

Adopt

Python. Остается стандартом для ML-сервисов из-за богатой экосистемы библиотек и высокой скорости разработки (Time-to-Market)

React. Сохраняется как основной стандарт для фронтенда.

Применение технологического радара для трансформации

Шаг 1. Снижение техдолга (Зона Hold): Архитектор идентифицирует Java-бэкенд как узкое место. Принимается решение: новые модули логики больше не пишутся на Java. Это высвобождает фокус команды для перехода Go.

Шаг 2. Пилотный проект (Зона Trial): Выбирается один изолированный сервис. Он переписывается на Go и разворачивается в Kubernetes.

Защита решения: Архитектор обосновывает переход снижением потребления оперативной памяти (в 5-10 раз по сравнению с JVM) и упрощением масштабирования в K8s.

Шаг 3. Фильтр экономики (ROI): Любое перемещение в зону Adopt (например, полный переход на K8s) оценивается по способности снижать издержки на поддержку (Ops) и ускорять выпуск новых функций (TTM). Если K8s позволяет быстрее обновлять ML-модели без простоя производства, он становится новым стандартом.

Резюме кейса

Использование радара превращает «хаотичное внедрение Go и K8s» в управляемую стратегию

- Java планово выводится из эксплуатации (Hold).
- Go проходит путь от проверки концепции (Assess) до боевого применения в малом сервисе (Trial).
- Kubernetes становится целевым фундаментом для объединения Python и Go сервисов в единую, гибкую экосистему

Заполните, пожалуйста, опрос о занятии

Мы читаем все ваши сообщения
и берем их в работу 🖋️❤️

OTUS

